

面向小语言模型的动态循环与奖励驱动早退机制

Dynamic Looping with Reward-Driven Early Exit for Small Language Models

MiniMind Project — Technical Report

摘要:

循环计算 (looped computation) 通过在隐空间中重复执行推理模块, 为预训练语言模型提供了一种不增加参数量即可扩展测试时计算 (test-time compute, TTC) 的手段。LoopUS 等现有方法依赖一个预先设定的固定循环次数上限 N , 并在训练中仅通过置信度头 (q-head) 的辅助分类损失间接影响退出行为, 早退决策本身并不参与优化目标。本文针对参数量约 64M 的小语言模型提出两项改进: (i) 动态循环——彻底移除固定循环次数 N , 循环在训练与推理中均按样本独立运行, 直至置信度头判定"已足够"为止, 仅保留一个实际不构成约束的安全上限; (ii) 奖励驱动的早退训练——将期望循环深度 $\mathbb{E}[\text{steps}]$ 以可微的存活链 (survival chain) 形式纳入训练损失, 通过 $\lambda \cdot \mathbb{E}[\text{steps}]$ 项直接为"更快退出"提供梯度信号, 使模型在保持预测质量的同时学会最小化计算开销。我们在 MiniMind (64M Dense, 自训练) 上验证了核心机制: 加载真实预训练权重后微调, 平均循环步数随训练进程从接近安全上限 (约 9.5) 收敛至 1, 同时总损失同步收敛 (0.106 ± 0.009), 证实了"训练越充分、推理越早退"的预期行为。该方法与 LayerSkip/PonderNet 等家族方法的不同之处在于: 退出决策与表征学习联合优化, 早退步数由训练习得, 而非依赖推理时的阈值精细调节。

关键词: 循环语言模型; 动态推理; 早退机制; 测试时计算; 小语言模型

1 引言

大语言模型的推理开销与其能力往往正相关。自回归解码过程中, 每个 token 都要经过全部 Transformer 层, 计算量与模型深度线性相关。然而并非所有 token 都需要同等深度的处理: 简单 token 在浅层即可被高置信度地预测, 而困难 token 需要更深的语义精炼 [1, 2]。这一观察催生了早退 (early exit) [3]、动态深度 (dynamic depth) [4] 与自适应计算时间 (adaptive computation time, ACT) [5] 等一系列动态计算范式。

循环语言模型 (looped language model) 是其中一类特殊的深度扩展方式: 不新增参数, 而是将模型中间的一个推理模块 (reasoning block) 反复执行 N 次, 在隐空间中迭代精炼表征 [6, 7, 8]。LoopUS [9] 是最新的代表工作之一, 它通过块分解将预训练 LLM 拆为 encoder / reasoning / decoder 三块, 引入选择性门 (Mamba 风格 SSM 衰减) 抑制隐状态漂移, 并用随机深度监督避免长循环上的反向传播爆炸。推理时, 其置信度头 (q-head) 在固定步数内评估是否需要提前停止。

尽管 LoopUS 已具备早退能力, 但其设计仍存在两点面向小模型时尤为突出的局限:

1. 固定循环预算 N 。循环步数上限在训练前设定, 与输入难度无关。训练时所有样本均执行满 N 步, 早退仅在推理时生效——训练与推理之间存在行为不一致 (train-inference gap)。对小模型而言, 这种不一致会被放大, 因为小模型可学习的表征空间有限, 训练时"从不退出"会抑制其形成"何时该停"的判别能力。
2. 早退未进入优化目标。LoopUS 通过辅助损失 $L_Q = \text{BCE}(q, \text{accuracy})$ 训练 q-head 预测"当前步预测是否准确", 但退出决策本身 (在哪一步停) 并不产生梯度。模型没有直接的信号去学

习"更早地退出"。对需要追求极致效率的小模型部署场景，这一缺失使得早退步数只能依赖推理时的阈值调节，而非训练习得。

本文针对上述两点，提出动态循环 + 奖励驱动早退（Dynamic Looping with Reward-Driven Early Exit），并在自研的 MiniMind（64M Dense）上实现与验证。我们的核心贡献如下：

- 动态循环：移除固定 N 。训练与推理中，每个样本独立循环，直至 $q \geq q_{th}$ 或到达安全上限 cap （实际上不构成约束）。
- 可微深度奖励：以存活链 $S_b = \prod_{j < b} (1 - q_j)$ 累积期望循环深度 $\mathbb{E}[\text{steps}] = \sum_b S_b$ ，将 $\lambda \cdot \mathbb{E}[\text{steps}]$ 加入训练损失。该梯度信号直接鼓励"更快退出"，与语言建模损失联合优化，使早退步数由训练习得而非靠推理阈值调参。
- 验证的涌现行为：加载真实预训练权重微调后，平均循环步数随训练收敛（约 $9.5 \rightarrow 1.0$ ），总损失同步下降，证实"训练越充分、推理越早退"。

2 相关工作

早退机制（Early Exit）。BranchyNet [3] 首次在深层网络侧分支上引入早退；DeeBERT [10] 与 FastBERT [11] 将早退引入 BERT，通过在中间层附加分类头并比较熵阈值决定是否提前输出。PABEE [12] 以"连续 K 层预测不变"作为停止条件。这些方法均以手动设定的阈值作为退出判据，退出决策不参与训练。

可微自适应计算。ACT [5] 以可微的"停顿单元"（halting unit）学习何时停止，PonderNet [13] 将停止建模为几何分布并施加 KL 正则。二者的停止决策是计算图的一部分，可直接反向传播——这是本文深度奖励的直接理论来源。CALM [1] 面向自回归生成引入序列级置信度约束。但这些方法要么针对非语言任务（ACT），要么需要从零训练（PonderNet）。

循环语言模型。Universal Transformer [6] 提出循环 Transformer 的概念。RTR [14]（retrofitted recurrence）、MoR [7]（mixture-of-recursions）等将循环引入预训练模型。LoopUS [9] 是其中系统化最强的工作之一：块分解 + 选择性门 + 随机深度监督 + q-head 早退。我们的工作直接以 LoopUS 为基线，移除其固定预算并使其早退决策参与训练。

与 PonderNet 的区别。PonderNet 学习一个隐式的停止分布，但其目标是从零训练的循环网络。我们面向已预训练的小模型，在保留预训练能力的前提下，通过可微的期望深度项让退出决策随训练涌现——这在 post-training 场景下是 LoopUS 家族特有的贡献。

3 方法

3.1 预备：LoopUS 的结构

LoopUS 将预训练 LLM 分解为三块：encoder $\mathcal{E}$ （前若干层，执行一次）、reasoning block $\mathcal{M}$ （中间层，循环执行）、decoder $\mathcal{D}$ （末层 + 最终归一化 + lm_head，执行一次）。第 b 次循环：

$$h_{b+1} = \mathcal{G}(\mathcal{M}(h_b), h_b) = \alpha_b \odot \mathcal{M}(h_b) + (1 - \alpha_b) \odot h_b$$

其中 $\mathcal{G}$ 是选择性门， $\alpha_b \in (0, 1)$ 由 Mamba 风格的输入相关衰减计算。推理时，置信度头 q_ϕ 在固定 N 步内评估 $q_b = \sigma(q_\phi(\text{norm}(h_b)))$ ，当 $q_b \geq q_{\text{th}}$ 时提前停止。

3.2 动态循环（Dynamic Looping）

移除固定预算。LoopUS 训练时无论输入如何都执行满 N 步；本文改为按样本独立决定步数。设安全上限 cap （默认 32，训练充分后实际不触及），每个样本 i 维护活跃标志 $a_b^{(i)}$ ：

$$a_{b+1}^{(i)} = a_b^{(i)} \wedge (q_b^{(i)} \leq q_{\text{th}})$$

循环在批内所有样本均退出（ $\sum_i a_b^{(i)} = 0$ ）时终止。具体地，样本在 $q_b^{(i)} > q_{\text{th}}$ 时退出（与代码中 `q.detach() > q_threshold` 的严格大于判据一致），退出样本的隐状态被冻结（`torch.where` 保留其最终状态），不再参与后续循环。这一设计使训练与推理行为一致：训练中模型即学会"在何处停"，消除 train-inference gap。

3.3 奖励驱动的早退训练（Reward-Driven Early Exit）

核心思想：退出决策应当参与优化。我们将"期望循环深度"作为可微的惩罚项加入训练目标。

定义第 b 步的存活概率（survival probability）：

$$S_b = \prod_{j < b} (1 - q_j), \quad S_0 = 1$$

S_b 表示"模型在步 b 之前尚未退出"的概率。期望循环深度即为存活链之和：

$$\mathbb{E}[\text{steps}] = \sum_{b=0}^{\text{cap}-1} S_b$$

该式对 q_ϕ 的梯度为：

$$\frac{\partial \mathbb{E}[\text{steps}]}{\partial q_j} = - \sum_{b=j+1}^{\text{cap}-1} \prod_{k < b, k \neq j} (1 - q_k) \leq 0$$

即提高任何一步的退出概率都会降低期望深度——这正是"鼓励更快早退"的梯度信号。由于 q 是 h_b 的函数，梯度经由 q_ϕ 、选择性门与推理模块回传，使模型在保持表征质量的同时学会在恰当深度退出。

训练目标。总损失为：

$$\mathcal{L} = \underbrace{\frac{1}{K} \sum_{b \in \mathcal{S}} \left[\mathcal{L}_{\text{LM}}^{(b)} + \beta \mathcal{L}_{\text{mono}}^{(b)} + \mathcal{L}_{Q^{(b)}} \right]}_{\text{质量项}} + \underbrace{\lambda \mathbb{E}[\text{steps}]}_{\text{深度奖励项}}$$

其中 $\mathcal{S}$ 为随机深度监督采样的步集合（每步独立以 β 权重施加单调性正则）， λ 为深度奖励权重。随机深度监督 [9] 保证仅 $|\mathcal{S}|$ 步回传梯度，避免长循环上反向传播的显存爆炸；而深度奖励项在整个循环上累积，为所有步的 q 提供梯度。

3.4 深度奖励的退火（Annealing）

早期训练中，模型尚未形成可靠的置信度估计，若 λ 过大，模型会过早退出而放弃精炼，导致"探索不足"的恶性循环。我们支持对 λ 退火：训练初期 λ 较小（鼓励充分循环、学习表征），随训练进度增大（加速退出）。§4.2 实验观察到步数下降与损失收敛同步发生，这与"准确率上升 $\rightarrow$ q 上升 $\rightarrow$ 更早退出"的机制一致： `set_depth_reward()` 接口使退火调度可由训练脚本灵活控制。需要说明的是，本文实验尚未严格分离"模型能力提升"与" λ 退火"对步数下降的各自贡献，此为后续工作方向。

4 实验

4.1 实验设置

基座：MiniMind-3 Dense，64M 参数，8 层 Transformer，hidden size 768，词表 6400。加载公开预训练权重 `pretrain_768.pth`。

循环结构：encoder = 层 [0,1]；reasoning = 层 [2,3,4]；decoder = 层 [5,6,7]。安全上限 $\text{cap} = 10$ ，阈值 $q_{\text{th}} = 0.75$ ，随机深度监督 $|S| = 5$ ， $\beta = 0.5$ ， $\lambda = 0.1$ ，`exit_in_training=True`。

数据：以 MiniMind tokenizer 编码的重复中文文本（机器学习/自然语言主题），序列长 48，batch 4，AdamW（lr=2e-4），60 次迭代。§4.2 报告的总损失为 §3.3 定义的训练目标 $\mathcal{L}$ （含深度奖励项 $\lambda \cdot \mathbb{E}[\text{steps}]$ ），而非纯语言建模损失。

对照设置：(a) 加载预训练权重、不训练；(b) 同上但开启动态循环 + 深度奖励训练。

4.2 核心结果：循环步数随训练递减

表 1 报告了 5 个随机种子下的均值 $\pm$ 标准差（每次迭代在固定中文重复文本上微调，配置同 §4.1）。`iter 0` 表示仅加载预训练权重、不做任何训练时的行为。

训练迭代	总损失 (mean $\pm$ std)	平均循环步数 (mean $\pm$ std)
0（仅预训练）	—	9.55 $\pm$ 0.90
1	8.747 $\pm$ 0.072	9.55 $\pm$ 0.90
5	4.443 $\pm$ 0.329	7.30 $\pm$ 2.62
10	2.162 $\pm$ 1.040	6.40 $\pm$ 3.37
20	0.272 $\pm$ 0.220	1.00 $\pm$ 0.00
30	0.205 $\pm$ 0.063	1.00 $\pm$ 0.00
60	0.106 $\pm$ 0.009	1.00 $\pm$ 0.00

表 1：5 个种子下，总损失（含深度奖励项）与平均循环步数随训练迭代的变化。数值为均值 $\pm$ 标准差。

观察：训练初期（`iter 1`）模型平均执行约 9.5 步（接近安全上限 10），此时总损失尚未收敛。随着训练推进，损失下降，模型学会在第 1 步即达到置信度阈值：从 `iter 20` 起，全部 5 个种子均在第 1 步退出（均值 1.00，标准差 0.00），并在后续训练中保持稳定。

需要诚实指出的是：中间阶段（`iter 5–10`）的步数与损失逐种子振荡（步数标准差高达 ± 3.4 ），这是因为随机深度监督采样与 `batch` 内样本的退出顺序引入了方差；但收敛后的行为（`iter  $\geq 20$` ）在所有种子上完全一致。循环步数的下降是训练自然涌现的行为，而非外部调参的结果。

4.3 消融与机制验证

深度奖励的可微性。我们对训练损失（含 $\lambda \cdot \mathbb{E}[\text{steps}]$ 项）求关于 q_ϕ 线性层权重的梯度，确认 `q_head` 参数梯度非零且更新方向正确：在 `depth_reward=1.0` 配置下，以学习率 0.1 执行一次梯度步后，`q_head` 线性层权重范数变化 1.64。这证明深度奖励的梯度确实经由存活链（§3.3）流回退出决策参数，而非仅作用于语言建模路径。

权重加载正确性。修复了 LoopUS 适配中的层映射 bug（`partition` 位置索引与原始层号混淆），使 91 个参数张量正确加载。仅跳过的 8 个张量全部来自新增模块：选择性门 4 个（`A_log`、`dt_input_proj.weight`、`delta_proj.weight`、`delta_proj.bias`）与置信度头 4 个（LayerNorm 的 `weight/bias` 与线性层 `weight/bias`），这些随机初始化的新模块随训练学习。

基线对照：加载预训练权重后，若不训练、仅推理，5 个种子下平均执行 9.55 ± 0.90 步（初始置信度不足，接近安全上限 10）。仅当训练推进后步数才下降——排除了“步数少是偶然”的可能。

5 讨论与局限

与 LoopUS 的对比总结：

维度	LoopUS	本文
循环预算	固定 N	动态，按样本独立，仅安全上限
训练/推理一致性	训练跑满 N ，推理才早退	训练与推理均动态退出
早退是否参与优化	否（仅辅助损失）	是（ $\lambda \cdot \mathbb{E}[\text{steps}]$ 可微项）
步数如何下降	依赖推理阈值调参	训练自然涌现
阈值 q_{th} 的作用	决定早退步数（需精细调节）	仅作停止条件；步数由训练习得（调节敏感性更低）

局限：(i) 目前实验在小规模、重复文本上验证，尚未在标准 benchmark（MMLU / C-Eval 等）上系统评测；(ii) `generate()` 因动态步数导致 per-depth KV cache 错位，采用每 token 全量重算，长序列生成效率有待优化（可通过缓存对齐策略恢复）；(iii) λ 的退火调度目前由外部脚本控制，未学习自适应调度。

未来工作：(i) 在完整预训练数据上验证，并评测“相同质量下平均循环步数”随训练数据量的缩放曲线；(ii) 将深度奖励与 RL（如 GRPO）结合，以序列级奖励替代 token 级准确率作为 q 的目标；(iii) 恢复 KV cache 的缓存对齐机制。

6 结论

本文针对小语言模型，在 LoopUS 基础上提出动态循环与奖励驱动的早退训练。通过移除固定循环预算并以可微的期望深度项 $\lambda \cdot \mathbb{E}[\text{steps}]$ 直接优化退出决策，模型在训练中自然学会随能力提升而

更早退出。实验验证了核心机制：加载预训练权重后，平均循环步数随训练从接近安全上限收敛至 1，同时总损失同步下降。该方法为小模型的测试时计算自适应提供了一条早退步数由训练习得、训练-推理一致的路径。

参考文献

1. [1] T. Schuster et al. *Confident Adaptive Language Modeling*. NeurIPS 2022. (CALM)
 2. [2] M. Elhoushi et al. *LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding*. ACL 2024.
 3. [3] S. Teerapittayanon et al. *BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks*. ICPR 2016.
 4. [4] D. Raposo et al. *Mixture-of-Depths: Dynamically allocating compute in transformer-based language models*. 2024.
 5. [5] A. Graves. *Adaptive Computation Time for Recurrent Neural Networks*. 2016.
 6. [6] M. Dehghani et al. *Universal Transformers*. ICLR 2019.
 7. [7] S. Bae et al. *Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation*. 2025.
 8. [8] R. Zhu et al. *Scaling Latent Reasoning via Looped Language Models*. 2025.
 9. [9] T. Park et al. *LoopUS: Recasting Pretrained LLMs into Looped Latent Refinement Models*. 2026.
 10. [10] J. Xin et al. *DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference*. ACL 2020.
 11. [11] W. Liu et al. *FastBERT: a Self-distilling BERT with Adaptive Inference Time*. ACL 2020.
 12. [12] W. Zhou et al. *BERT Loses Patience: Fast and Robust Inference with Early Exit*. NeurIPS 2020.
 13. [13] A. Banino et al. *PonderNet: Learning to Ponder*. ICML 2021.
 14. [14] S. Bae et al. *Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA*. ICLR 2025.
-

注：本论文基于 MiniMind 仓库中 `model/model_looped_minimind.py` 的实际实现撰写；§4.2 实验结果来自真实预训练权重（`pretrain_768.pth`）上的微调验证，可复现。